B.Sc. Information Technology — Semester VI

Software Testing

Course Code: 25UBIT604

UNIT 1

Basics of Software Testing

Topics Covered:

① Software Testing Fundamentals
② Verification vs. Validation
③ SDLC vs. STLC
④ Types of Testing: Manual, Automation, Regression, Smoke, Sanity, Ad-hoc
⑤ Defect Life Cycle
⑥ Severity and Priority

Exam Starts: 24th March | Credits: 4 | Max Marks: 150

1. Software Testing Fundamentals

1.1 What is Software Testing?

Software Testing is the process of evaluating and verifying that a software application or system meets specified requirements and works as expected. It involves executing a program or system with the intent of finding errors, bugs, or defects.

Simple Definition:

Testing = Finding bugs BEFORE the software reaches the end user.

1.2 Why is Software Testing Important?

Reason	Explanation	Real-World Example
Ensures Quality	Confirms software works as per requirements	Banking app correctly transfers money
Finds Bugs Early	Cheaper to fix bugs before release than after	Bug found in development costs 10x less than in production
Improves Security	Identifies security vulnerabilities	Finds SQL injection holes before hackers do
Customer Satisfaction	Reliable software = happy users	WhatsApp works without crashes
Saves Cost	Early bug detection = less rework and maintenance	NASA — software bugs can cost millions
Legal Compliance	Software must meet industry standards	Healthcare apps must meet HIPAA standards

1.3 Key Testing Terminologies

Term	Definition	Example
Bug / Defect	A flaw in the software that causes incorrect result	Login button not working
Error	Human mistake made during coding	Developer types wrong formula
Failure	System produces wrong output during execution	App crashes when user logs in
Test Case	A set of conditions to verify specific functionality	Test: Enter valid username + password → Should login
Test Plan	Document describing scope, approach, and schedule of testing	Plan for testing entire e-commerce website
Test Suite	Collection of related test cases	All login-related test cases grouped together
Test Script	Step-by-step instructions to execute a test	Selenium script to automate login test

1.4 Principles of Software Testing

- Testing shows presence of defects — NOT absence. Testing can never prove 100% bug-free
- Exhaustive testing is impossible — test all possible inputs is not feasible
- Early testing — find defects as early as possible in SDLC (cheaper to fix)
- Defect clustering — 80% of bugs found in 20% of modules (Pareto Principle)
- Pesticide Paradox — same tests repeated find no new bugs; update test cases regularly
- Testing is context dependent — testing approach changes based on type of application
- Absence of errors fallacy — bug-free software is useless if it doesn't meet user needs

---

2. Verification vs. Validation

2.1 Definitions

Both Verification and Validation are quality assurance activities in software development, but they answer different questions.

Aspect	Verification	Validation
Definition	Are we building the product RIGHT?	Are we building the RIGHT product?
Focus	Process-oriented — checks process and documents	Product-oriented — checks the actual software
Activity Type	Static testing — no code execution needed	Dynamic testing — requires code execution
When Done	During development (before testing)	After development (during/after testing)
Goal	Ensures software conforms to specifications	Ensures software meets user needs and expectations
Methods	Reviews, walkthroughs, inspections, audits	Unit testing, Integration testing, System testing, UAT
Who Does It	QA team, developers, reviewers	Testing team, end users
Example	Reviewing SRS document for completeness	Testing login functionality actually works
Cost if Failed	Cheaper to fix — caught early in process	Expensive to fix — caught late in development

2.2 Simple Memory Trick

Question	Answer	Type
Are we building the product RIGHT?	Correct process, correct design, correct specs	Verification
Are we building the RIGHT product?	Does it satisfy user needs and business goals?	Validation

> 📌 *Example: Verification = checking blueprint of a house is correct | Validation = checking if the built house actually meets owner's needs*

✅ **Key Point: Verification catches design errors. Validation catches requirement mismatches.**

---

3. SDLC vs. STLC

3.1 SDLC — Software Development Life Cycle

SDLC is the process used by software development teams to design, develop, and deliver high-quality software. It defines the complete lifecycle of a software product from planning to maintenance.

Phase	Activity	Output
1. Requirement Analysis	Gather and analyze business requirements	SRS (Software Requirement Specification)
2. System Design	Create architecture and technical design	Design Document (HLD, LLD)
3. Implementation/Coding	Developers write the actual code	Source Code
4. Testing	Find and fix bugs in the software	Test Reports, Bug Reports
5. Deployment	Release software to production environment	Live Application
6. Maintenance	Fix post-release bugs, add new features	Updated Software

3.2 STLC — Software Testing Life Cycle

STLC is a sequence of specific activities performed during the testing process to ensure software quality. It is a subset of SDLC focused entirely on testing activities.

Phase	Activity	Output/Deliverable
1. Requirement Analysis	Understand testable requirements, identify gaps	Requirement Traceability Matrix (RTM)
2. Test Planning	Define scope, strategy, schedule, resources	Test Plan Document
3. Test Case Design	Write detailed test cases and test scripts	Test Cases, Test Scripts, Test Data
4. Test Environment Setup	Configure hardware, software, test tools	Test Environment Ready
5. Test Execution	Execute test cases, log bugs, re-test fixes	Test Results, Bug Reports
6. Test Closure	Evaluate completion, prepare final report	Test Closure Report, Lessons Learned

3.3 SDLC vs STLC — Comparison

Feature	SDLC	STLC
Full Form	Software Development Life Cycle	Software Testing Life Cycle
Purpose	Complete software development process	Testing process only
Scope	Covers entire software development	Subset of SDLC — testing activities only
Team	Developers, designers, PMs, testers	QA engineers, testers
Start Point	Requirement gathering	Requirement analysis (for testing)

Feature	SDLC	STLC
End Point	Software deployment and maintenance	Test closure and reporting
Output	Working software product	Test reports, bug reports, quality sign-off

📌 *STLC runs parallel to SDLC — testing activities begin from requirement phase itself*

4. Types of Testing

4.1 Manual Testing

Manual Testing is the process where testers manually execute test cases without using any automation tool. The tester plays the role of an end user and verifies software behavior.

- Tester manually interacts with the application step by step
- No automated scripts — purely human-driven
- Best for exploratory, usability, and ad-hoc testing
- Time-consuming but flexible — can adapt to unexpected scenarios

📌 *Example: A tester manually fills a login form and checks if login works correctly*

4.2 Automation Testing

Automation Testing uses specialized software tools to execute pre-scripted test cases automatically. It is best for repetitive, regression, and performance testing.

- Uses tools like Selenium, JUnit, TestNG, Postman
- Faster than manual testing for large test suites
- Reusable test scripts — run anytime without human effort
- Best for regression testing, smoke testing, and load testing

📌 *Example: Selenium script automatically opens browser, enters credentials, and verifies login*

4.3 Manual vs Automation Testing

Feature	Manual Testing	Automation Testing
Execution	Human executes test cases manually	Tool/script executes tests automatically
Speed	Slower — human dependent	Faster — runs 24/7 without breaks
Cost	Lower initial cost	Higher initial cost (tools + scripting)
Accuracy	Prone to human error	More accurate — same steps every time
Best For	Exploratory, usability, ad-hoc testing	Regression, smoke, load, repeated tests
Flexibility	Very flexible — adapts quickly	Less flexible — needs script updates
Tools	No specific tools needed	Selenium, JUnit, TestNG, Postman

4.4 Regression Testing

Regression Testing is performed after code changes (bug fixes, new features) to ensure that previously working functionality has NOT been broken. It re-runs existing test cases.

- Performed after every code change, bug fix, or new feature addition
- Ensures existing functionality still works correctly after modifications
- Usually automated to save time — large number of test cases
- Types: Full Regression, Partial Regression, Sanity Regression

📌 *Example: After adding 'Forgot Password' feature, re-test login, signup, and other existing features*

4.5 Smoke Testing

Smoke Testing (also called Build Verification Testing) is a preliminary test to check if the basic and critical functionalities of a new build are working. It decides if the build is stable enough for further testing.

- First test performed on a new software build
- Tests only critical/core functionalities — not detailed testing
- Quick test — takes 30 minutes to 2 hours
- If smoke test fails → build is rejected, sent back to developers

📌 *Analogy: Like turning on a new electronic device — if it smokes, don't test further!*

📌 *Example: After new build — check if app launches, login works, home page loads correctly*

4.6 Sanity Testing

Sanity Testing is a subset of regression testing performed after receiving a software build with minor changes or bug fixes. It checks if the specific bug is fixed and related functionality works correctly.

- Narrow and deep testing — focuses on specific area only
- Performed after regression testing or after a specific bug fix
- Quick check — not exhaustive like full regression
- Usually not scripted — done informally by testers

📌 *Example: If password reset bug is fixed, sanity test only tests password reset flow*

4.7 Smoke vs Sanity Testing

Feature	Smoke Testing	Sanity Testing
Purpose	Check overall build stability	Check specific bug fix or feature
Scope	Broad — covers entire application briefly	Narrow — focuses on specific module
When Done	After receiving a new build	After bug fix or minor change
Performed By	Testers or developers	Testers
Documentation	Usually scripted	Usually unscripted (informal)
If Fails	Build rejected — not tested further	Build sent back for specific fix

4.8 Ad-hoc Testing

Ad-hoc Testing is an informal and unplanned testing approach where testers test the application randomly without following any test cases, test plans, or documentation. The goal is to find defects that formal testing may miss.

- No formal test cases or test plans — completely random
- Requires deep knowledge of the application
- Relies on tester's experience and intuition
- Types: Buddy Testing, Pair Testing, Monkey Testing

📌 *Example: Tester randomly clicks buttons in an application to find unexpected crashes*

4.9 Quick Summary — All Testing Types

Testing Type	Purpose	When Used
Manual Testing	Human-driven test execution	Exploratory, usability, new features
Automation Testing	Tool-driven automatic execution	Regression, smoke, repetitive tests
Regression Testing	Ensure old features still work after changes	After every code change / bug fix
Smoke Testing	Verify build stability — basic functions work	After receiving new build
Sanity Testing	Verify specific bug fix works correctly	After bug fix or minor change
Ad-hoc Testing	Random unplanned testing to find hidden bugs	Anytime during testing phase

5. Defect Life Cycle

5.1 What is a Defect Life Cycle?

Defect Life Cycle (also called Bug Life Cycle) is the journey of a defect from the time it is discovered by a tester to the time it is closed after being fixed and verified. It defines the various states a defect goes through.

5.2 Defect Life Cycle Stages

Stage	Who Acts	Description
New	Tester	Defect is discovered and reported for the first time
Assigned	Test Lead / Manager	Defect is assigned to a developer for investigation
Open	Developer	Developer starts analyzing and working on the fix
Fixed	Developer	Developer fixes the bug and marks it as Fixed
Retest	Tester	Tester re-executes the test case to verify the fix
Verified	Tester	Tester confirms the bug is fixed — passes the retest
Closed	Tester / Manager	Bug is confirmed fixed — defect is officially closed
Rejected	Developer	Developer rejects — not a bug or duplicate
Deferred	Manager	Bug fix postponed to a future release
Reopened	Tester	Bug reappears after fix — defect is reopened

5.3 Defect Life Cycle Flow (Text Diagram)

Flow	Path
Normal Path	New → Assigned → Open → Fixed → Retest → Verified → Closed
Rejected Path	New → Assigned → Rejected → Closed
Deferred Path	New → Assigned → Deferred (fixed in next release)
Reopened Path	Closed → Reopened → Assigned → Fixed → Retest → Closed

5.4 Defect Report — Key Fields

Field	Description	Example
Defect ID	Unique identifier for the bug	BUG-001
Title	Short description of the bug	Login button not responding on click
Description	Detailed steps to reproduce the bug	1. Open app 2. Enter credentials 3. Click Login — Nothing happens
Severity	Impact of the bug on the system	Critical
Priority	Urgency of fixing the bug	High

Field	Description	Example
Status	Current state in defect life cycle	Open
Assigned To	Developer responsible for the fix	Dev Team Member Name
Environment	Where bug was found	Windows 10, Chrome 120, Staging Server

---

6. Severity and Priority

6.1 What is Severity?

Severity describes the impact of a defect on the functionality of the application. It is determined by the tester based on how much the defect affects the system's behavior.

Severity Level	Description	Example
Critical (S1)	Application crashes or data loss — cannot proceed	App crashes on login — no one can use the system
Major (S2)	Major feature not working — workaround difficult	Payment gateway fails — orders cannot be placed
Medium (S3)	Feature partially working — workaround available	Search filter not working — user can still browse manually
Minor (S4)	Cosmetic issue — doesn't affect functionality	Button color is wrong — app still works fine
Trivial (S5)	Very minor — grammar or spelling error	Typo in non-critical label text

6.2 What is Priority?

Priority describes how urgently a defect needs to be fixed. It is determined by the project manager or business team based on business impact and deadlines.

Priority Level	Description	Example
High (P1)	Must be fixed immediately — cannot release without fix	Login broken — users cannot access the system
Medium (P2)	Should be fixed in current release if possible	Report export gives wrong format
Low (P3)	Can be fixed in future release — not urgent	Minor UI alignment issue on settings page

6.3 Severity vs Priority — Comparison

Feature	Severity	Priority
Definition	Impact of defect on system functionality	Urgency of fixing the defect
Set By	Tester / QA Engineer	Project Manager / Business Analyst
Based On	Technical impact on the application	Business impact and deadlines
Changes?	Usually doesn't change	Can change based on business needs

6.4 Severity vs Priority — All 4 Combinations

Combination	Example	Action
High Severity + High Priority	App crashes on login — everyone affected	Fix immediately — top priority
High Severity + Low Priority	Crash in rarely used admin report module	Fix eventually — not urgent business-wise
Low Severity + High Priority	Company logo missing on homepage — brand issue	Fix quickly — business visibility matters
Low Severity + Low Priority	Spelling error on help page	Fix when time permits — least urgent

📌 *High Severity ≠ High Priority always! A critical bug in a rarely used feature may have low priority*

✅ **Key Point: Severity = How bad is the bug technically? | Priority = How fast must it be fixed?**

Unit 1 — Quick Revision Summary

Topic	Key Points
Software Testing	Finding bugs before release | Ensures quality, security, reliability
Key Terms	Bug=defect | Error=human mistake | Failure=wrong output | Test Case=conditions to verify
7 Principles	Shows presence of defects | Exhaustive testing impossible | Early testing | Defect clustering | Pesticide paradox | Context dependent | Absence of errors fallacy
Verification	Are we building product RIGHT? | Static | Process-oriented | Reviews, inspections
Validation	Are we building RIGHT product? | Dynamic | Product-oriented | Actual testing
SDLC Phases	Requirement → Design → Coding → Testing → Deployment → Maintenance
STLC Phases	Req Analysis → Test Planning → Test Design → Env Setup → Execution → Closure
Manual Testing	Human-driven | Flexible | Exploratory | Slower | No tools needed
Automation Testing	Tool-driven | Faster | Reusable scripts | Selenium, JUnit, TestNG
Regression Testing	Re-test after code changes | Ensures old features still work
Smoke Testing	Broad quick test on new build | Basic functions | Build accepted/rejected
Sanity Testing	Narrow test after specific fix | Checks that specific fix works
Ad-hoc Testing	Random unplanned | No test cases | Experience-based | Monkey testing
Defect Life Cycle	New→Assigned→Open→Fixed→Retest→Verified→Closed | Also: Rejected/Deferred/Reopened
Severity	Impact on system | Set by tester | Critical/Major/Medium/Minor/Trivial
Priority	Urgency of fix | Set by manager | High/Medium/Low | Based on business needs

Important Exam Questions

- What is Software Testing? Why is it important? Explain with examples (5 marks)
- Explain the 7 principles of Software Testing (7 marks)
- Differentiate between Verification and Validation with examples (5 marks)
- Explain SDLC vs STLC with all phases and comparison (7 marks)
- What is Regression Testing? How is it different from Smoke and Sanity Testing? (5 marks)
- Explain the Defect Life Cycle with all stages and flow diagram (7 marks)
- What is Severity and Priority? Explain all 4 combinations with examples (5 marks)
- Differentiate between Manual Testing and Automation Testing (5 marks)
- What is Ad-hoc Testing? Explain its types (2 marks)

✅ **All the best! Software Testing Unit 1 — Basics of Software Testing — Fully Covered!** 🚀